HTTP Headers & Caching

What each header does, which ones pair up, and how the browser decides between `304` and a fresh `200`.

1 `Cache-Control` runs in two directions

The same header name appears on both sides of the exchange carrying different meanings. It is not one shared setting negotiated between them — it is two separate instructions aimed at different parties.

Browser cache
private — only you
on your disk

→

CDN / proxy
shared — many users
one copy serves all

→

Server
the origin
stores nothing

"Caches in the path" means everything to the left of the server. A request travels left to right and each cache reads it in turn. All `Cache-Control` numbers are seconds — `604800` is 7 days.

ON THE REQUEST — A CEILING

Client → the caches in front

"Do not hand me a stored copy older than this." It sets the client's tolerance; it cannot make something cacheable that the server refused to allow.

`max-age=60` a copy up to 60s old is fine, older than that go check
`max-age=0` the same rule with zero tolerance — always check
`no-cache` always check, stated explicitly

ON THE RESPONSE — A GRANT

Server → whoever stores it

"Here are the terms on which you may keep this." Sets how long the copy counts as fresh and who is allowed to hold it.

`max-age=604800` fresh for 7 days (604800 seconds)
`no-cache` store it, but check with the server before every use
`private` the browser may store it, the CDN may not
`no-store` never write it down — used for authenticated pages

> **Easy to miss:** `no-cache` does not mean "do not cache". It means "cache, but check every time". The one that forbids storage is `no-store`.

2 Freshness decides *whether* to ask; validation decides *what comes back*

Two separate mechanisms. **Freshness** is whether the stored copy is still inside its `max-age` — past that it is **stale**. **Validation** is asking the server about a stale copy using the **validator**, the version marker the server sent with the body. `Cache-Control` never produces a `304` by itself; it only decides whether a request is sent at all.

FRESH

No request is sent.

Age is under `max-age`. The browser serves from memory or disk.

DevTools: *(from disk cache)*, 0 ms, no network row.

STALE, UNCHANGED

Request sent → `304`.

Validator matched. Headers only, no body. Freshness clock restarts.

One round trip, a few hundred bytes, the old body reused.

STALE, CHANGED

Request sent → `200`.

Validator did not match. Full body plus a new `ETag`.

Full transfer, cache entry replaced.

A `304` still costs a round trip. Long `max-age` is what removes the request; validation only removes the body.

3 Validators on the wire

The server hands out a validator with the body. The browser hands it back to ask whether it is still current. Two pairs exist, and they are not equal.

SERVER SENDS	BROWSER RETURNS	NOTES
`ETag: "3f8a-1e2b"`	`If-None-Match`	Exact version id for this body. Detects any change, including two edits landing in the same second.
`Last-Modified: …GMT`	`If-Modified-Since`	Timestamp comparison, to the second. Weaker: two edits inside one second look identical.

Send both and the browser returns both; the server must prefer `If-None-Match`.

4 Navigation, reload, hard reload

The three ways you land on the same page send different request headers, which is why a bug disappears on refresh and returns for users.

ACTION	REQUEST CARRIES	RESULT
Normal navigation link, address bar	Nothing extra	Fresh copy served with no request. Stale copy revalidates.
Reload ⌘R / F5	**`Cache-Control: max-age=0`** plus the validator	Freshness skipped, validation still runs — usually **`304`**.
Hard reload ⇧⌘R, or DevTools open + Disable cache	**`Cache-Control: no-cache`**, validator *omitted*	Nothing left to match on, so a full **`200`** every time.

> **Common mistake:** testing a caching change with DevTools open and "Disable cache" ticked. Every request looks like a hard reload, so the behaviour you are trying to verify never runs.

5 Request headers, by job

HEADER	WHAT IT DOES
ROUTING & IDENTITY	
`Host`	Which site on this IP you want. Mandatory in HTTP/1.1 — one server hosts many domains. Host: bytebytego.com HTTP/2 sends this as **`:authority: bytebytego.com`** — see *A real exchange*
`User-Agent`	Client software string. Widely used for analytics and bot filtering. User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
`Referer`	The page that generated this request. Referer: https://bytebytego.com/courses
NEGOTIATION — WHAT YOU CAN ACCEPT BACK	
`Accept`	Media types wanted back in the *response*. The server answers with **`Content-Type`**. Accept: image/webp,image/avif,*/*
`Accept-Encoding`	Compression formats you can decode — **`br`** is Brotli. Accept-Encoding: gzip, br, zstd
STATE & AUTH	
`Cookie`	Everything previously set by **`Set-Cookie`** for this origin, sent back on every matching request. Cookie: session=a1b2c3; theme=dark
`Authorization`	Credentials — bearer token, Basic, etc. Authorization: Bearer eyJhbGciOiJI…
`Origin`	Scheme + host + port of the calling page. What the browser's cross-origin rules (CORS) are decided against. Origin: https://app.example.com
BODY — ONLY WHEN THERE IS ONE	
`Content-Type`	Media type of the body being *sent*. Opposite direction to **`Accept`**. Content-Type: application/json
CACHING	
`Cache-Control`	The client's ceiling on how old a stored copy may be. Cache-Control: max-age=0
`If-None-Match`	Returns the stored **`ETag`** to ask whether it is still current. If-None-Match: "3f8a-1e2b"

6 Response headers, by job

HEADER	WHAT IT DOES
BODY	
`Content-Type`	What the body actually is. Include the charset for text. Content-Type: text/html; charset=utf-8
`Content-Encoding`	Compression applied. The reply to **`Accept-Encoding`**. Content-Encoding: br
CACHING	
`Cache-Control`	The server's grant: who may store this, and for how many seconds. Cache-Control: max-age=604800
`ETag`	Version id for this exact body. Changes whenever the body changes. ETag: "3f8a-1e2b"
`Last-Modified`	When the body last changed. Weaker than **`ETag`** — only accurate to the second. Last-Modified: Mon, 01 Sep 2026 09:14:02 GMT
`Vary`	Extra request headers a cache must match on, so a Brotli copy is never handed to a client that cannot decode it. Vary: Accept-Encoding
STATE, CORS, CONTROL FLOW	
`Set-Cookie`	Stores a cookie on the client. One header per cookie. Set-Cookie: session=a1b2c3; HttpOnly; Secure; SameSite=Lax
`Access-Control-Allow-Origin`	Which origins the browser may expose this response to. Answers **`Origin`**. Access-Control-Allow-Origin: https://app.example.com
SECURITY	
`Strict-Transport-Security`	Use HTTPS for this host for the next N seconds, no exceptions. Strict-Transport-Security: max-age=31536000
`Content-Security-Policy`	Which sources of script, style and frames are allowed to run. Content-Security-Policy: default-src 'self'

7 A real exchange

One conditional request and its `304`, shown in both protocol versions. HTTP/2 has no request line and no status line. Those fields became **pseudo-headers** — names starting with a colon, which is reserved so they can never collide with a real header. Four exist for requests, one for responses, and they must come before any ordinary header.

HTTP/1.1

```
GET /courses HTTP/1.1
Host: bytebytego.com
Accept: image/webp
Cookie: session=a1b2c3

HTTP/1.1 304 Not Modified
ETag: "3f8a-1e2b"
```

HTTP/2

```
:method: GET
:scheme: https
:authority: bytebytego.com
:path: /courses
accept: image/webp
cookie: session=a1b2c3

:status: 304
etag: "3f8a-1e2b"
```

Header names are lowercase on the wire, and HPACK-compressed rather than sent as text — DevTools decodes them into the form above. Every header's meaning is unchanged across versions; only the framing differs.